



Bondly

Full-stack social media web app

Project Overview and UI Walkthrough

Bondly is my individual full-stack social media web application. I built it to connect real backend workflows with a usable React interface: registration, OTP verification, authentication, profiles, posts, social interactions, real-time notifications, support handling, and account recovery.

Spring Boot Backend

React Frontend

PostgreSQL

JWT Auth

SSE Realtime

What this document covers

This PDF explains the project from the outside user's point of view and from the developer side. It includes the stack I used, the main feature set, the backend and frontend structure, API areas, security notes, and a screenshot-based UI walkthrough.

2

Main applications

16

Frontend routes

13+

API areas

10

UI screenshots

Project Overview

Application idea

Bondly works like a small social network. A user can create an account, verify it through email OTP, login, update the profile, create posts, react, comment, share, follow users, manage friends, receive notifications, and ask support questions.

The project is not only a UI mockup. The frontend is connected to a Spring Boot REST backend with database persistence, authentication, upload handling, and service-layer business logic.

Main layers

React UI

Pages, navigation, forms, feed, profile, support, notifications.

API Client

Axios and EventSource connect the frontend to backend endpoints.

Spring API

Controllers expose auth, posts, user, support, social, and review routes.

Service Layer

Business logic for OTP, social actions, uploads, realtime events.

Database

PostgreSQL stores users, posts, comments, friends, follows, notifications.

Authentication

Registration uses email OTP verification. Login uses Spring Security, BCrypt, and JWT tokens. Password reset uses a separate OTP flow and a short-lived verification token.

Social features

Posts, image uploads, comments, replies, reactions, shares, followers, following, and friend requests are split into clear backend modules.

Realtime features

Notifications and support updates use Server-Sent Events, so active users can receive updates without manually refreshing the page.

Technology Stack

Backend

Java 25

Spring Boot 4.0.2

Spring Web MVC

Spring Security

Spring Data JPA

PostgreSQL

JWT / jjwt

BCrypt

Java Mail Sender

Server-Sent Events

Lombok

Maven

Frontend

React 19

Vite

React Router

Axios

Material UI

MUI Icons

Lucide React

Framer Motion

Styled Components

CSS page styling

Why this stack fits the project

Spring Boot gives the backend a structured controller-service-repository style. Spring Security and JWT handle authentication. JPA keeps database work manageable. React and Vite make the frontend fast to build and easy to organize into pages. Material UI, icons, and animation libraries help the interface feel more complete without writing every UI control from zero.

Backend Structure

Controllers

REST endpoints are grouped by feature: auth, user, posts, comments, reactions, shares, follows, friends, notifications, support, reviews, and chatbot.

Services

The service layer keeps the main logic away from controllers. It handles account flows, OTP logic, social actions, uploads, and realtime events.

Repositories

Spring Data repositories handle persistence for users, posts, comments, reactions, shares, follows, friends, notifications, reviews, and support questions.

Feature coverage

Email OTP registration and verification	JWT login with HTTP-only cookie support	BCrypt password hashing
Forgot-password OTP recovery	Profile and cover image upload	Text/image posts and feed
Comments, replies, and delete ownership checks	Multiple reaction types with counts	Post sharing with captions
Followers, following, user search	Friend request workflow	Real-time notifications with SSE
User/admin support flow	Reviews and ratings	Role-based dashboard routing

Frontend Routes and API Areas

Frontend route map

/	Landing page
/signup	Create account
/verify	OTP verification
/login	Login
/forgot-password	Forgot password
/dashboard	Admin dashboard
/home	Main feed
/profile	Profile
/settings	Account settings
/review	Reviews
/supportAdmin	Admin support
/supportUser	User support
/searchUsers	User search
/notifications	Notifications
/friend	Friends
/post	Post/share page

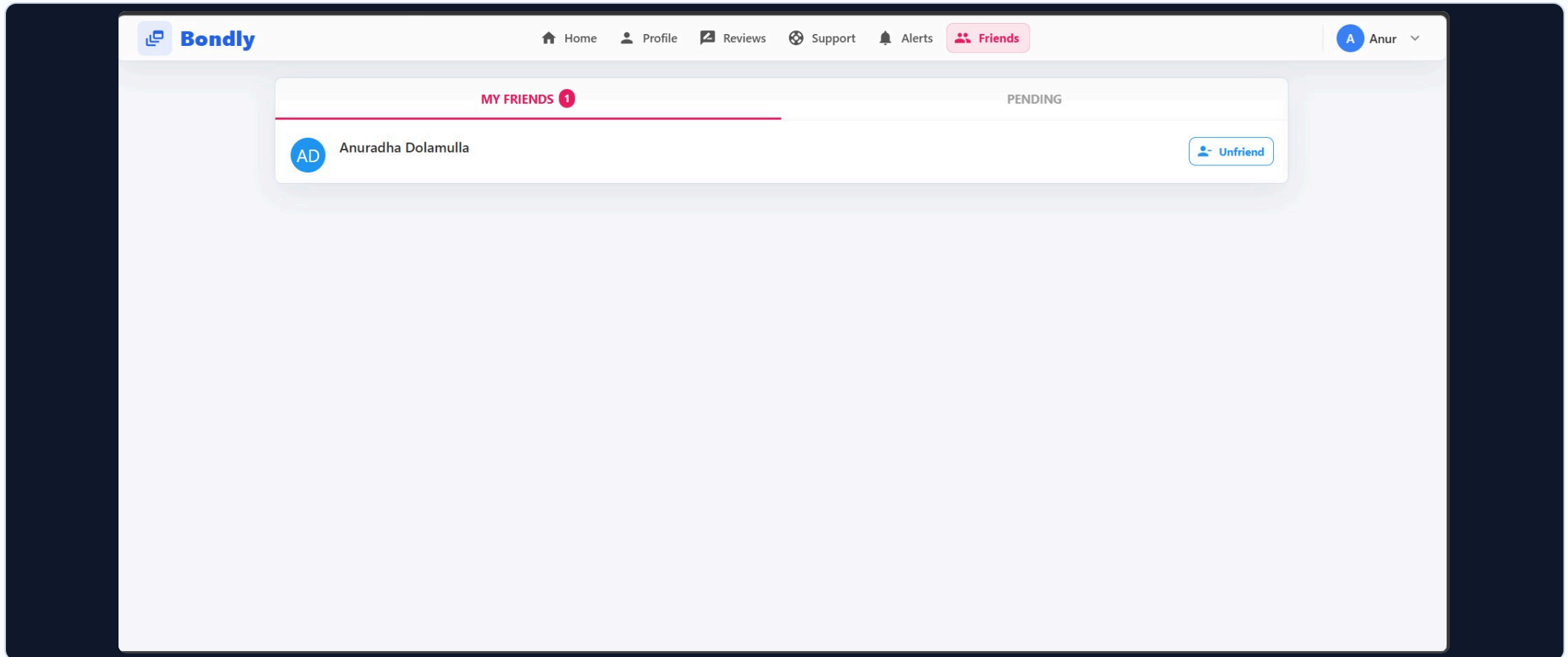
Backend endpoint groups

/auth	Register, login, verify account OTP, resend OTP	/forgotpass	Forgot password OTP and password reset
/user	Profile, settings, uploads, account deletion	/posts	Create posts, delete posts, personal posts, feed
/comments	Comments, replies, delete, counts	/reactions	Add/remove reactions and reaction counts
/shares	Share posts and shared feed	/follow	Follow/unfollow, followers, following, search
/api/friends	Friend requests and friend list	/notifications	Notifications and SSE stream
/support	User questions, admin responses, SSE stream	/reviews	Review CRUD
/api/chat	Rule-based and fuzzy concept chatbot		

01. Friends Management

UI walkthrough from the current Bondly interface.

Social Graph



Shows accepted friends and pending requests in a clean tab layout.

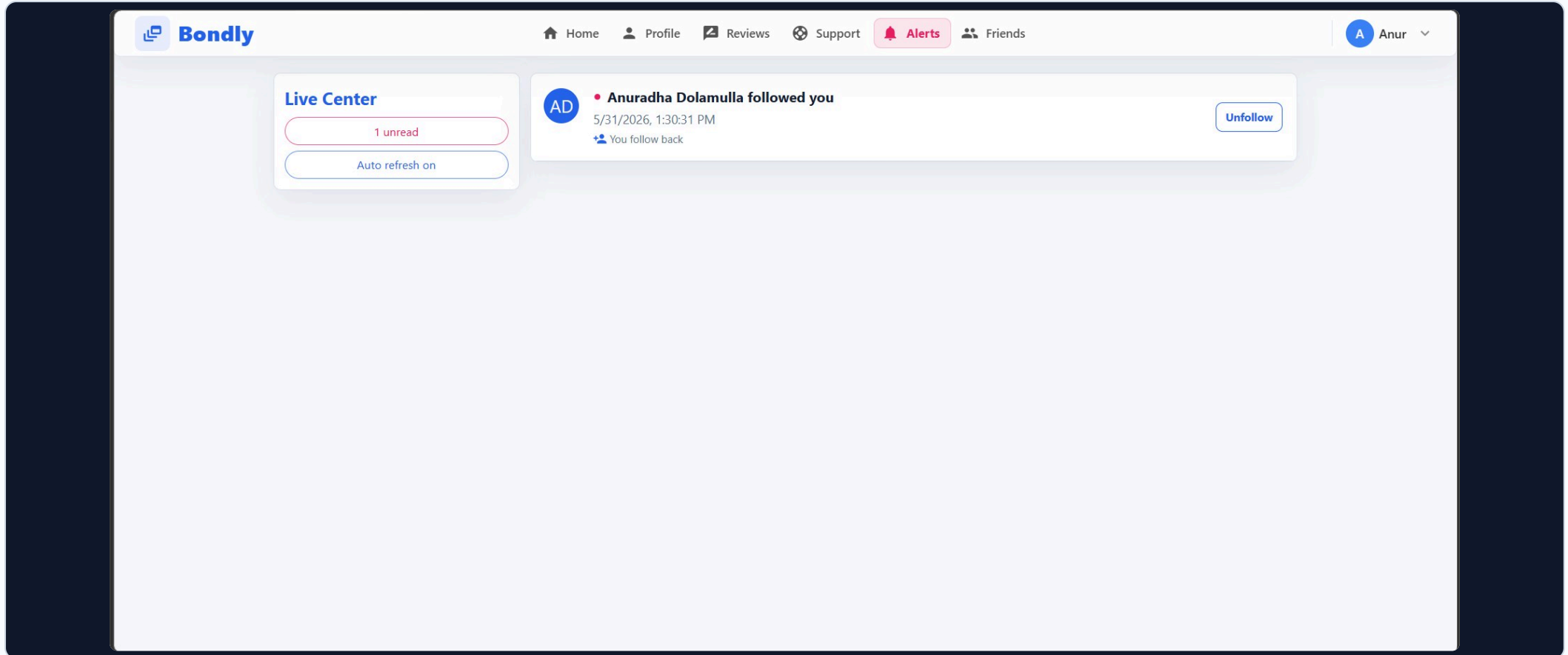
Friendship actions are handled from the backend through request, accept, reject, and unfriend endpoints.

The UI keeps the user action visible with a direct Unfriend button.

02. Real-time Alerts

UI walkthrough from the current Bondly interface.

SSE Notifications



The alert center receives follow and activity notifications from the backend.

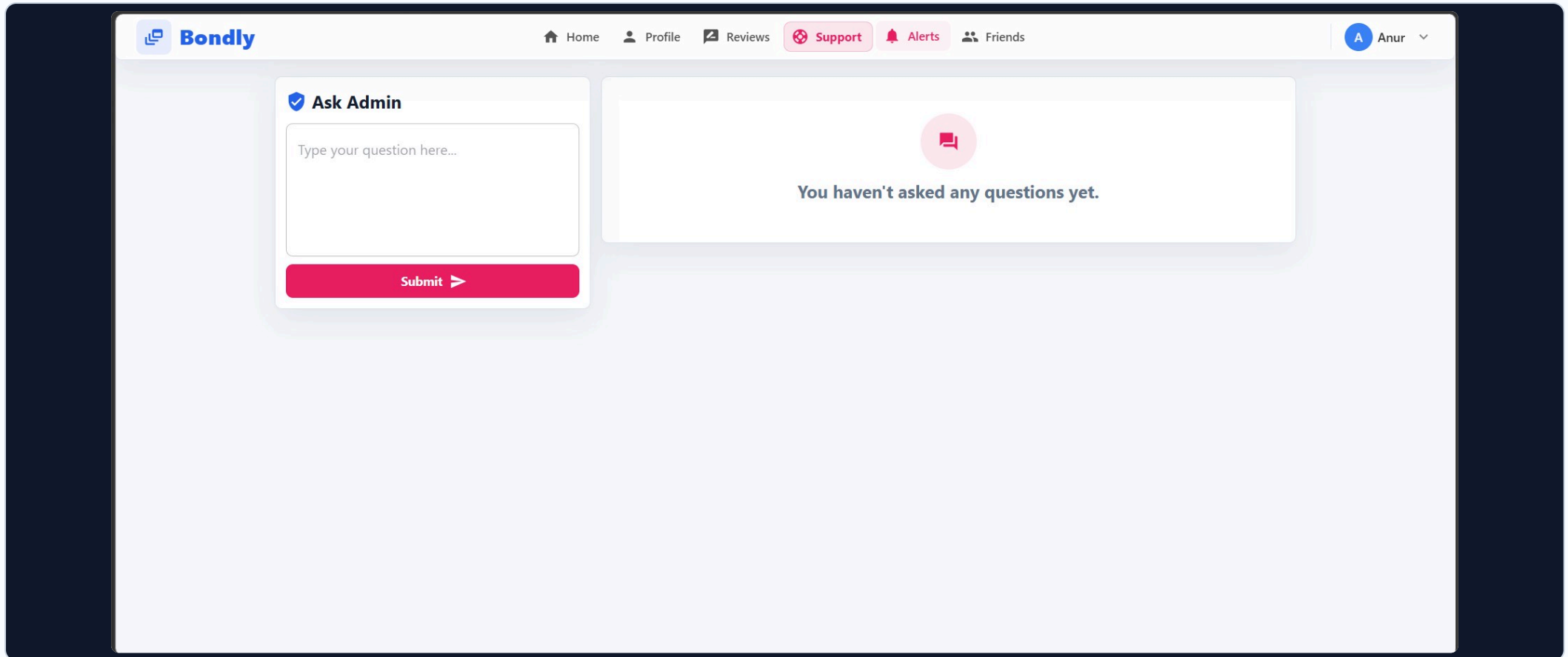
Server-Sent Events keep the page updated without a full refresh.

Follow/unfollow state is connected directly to the notification item.

03. User Support Panel

UI walkthrough from the current Bondly interface.

Support Flow



Users can submit a support question from inside the application.

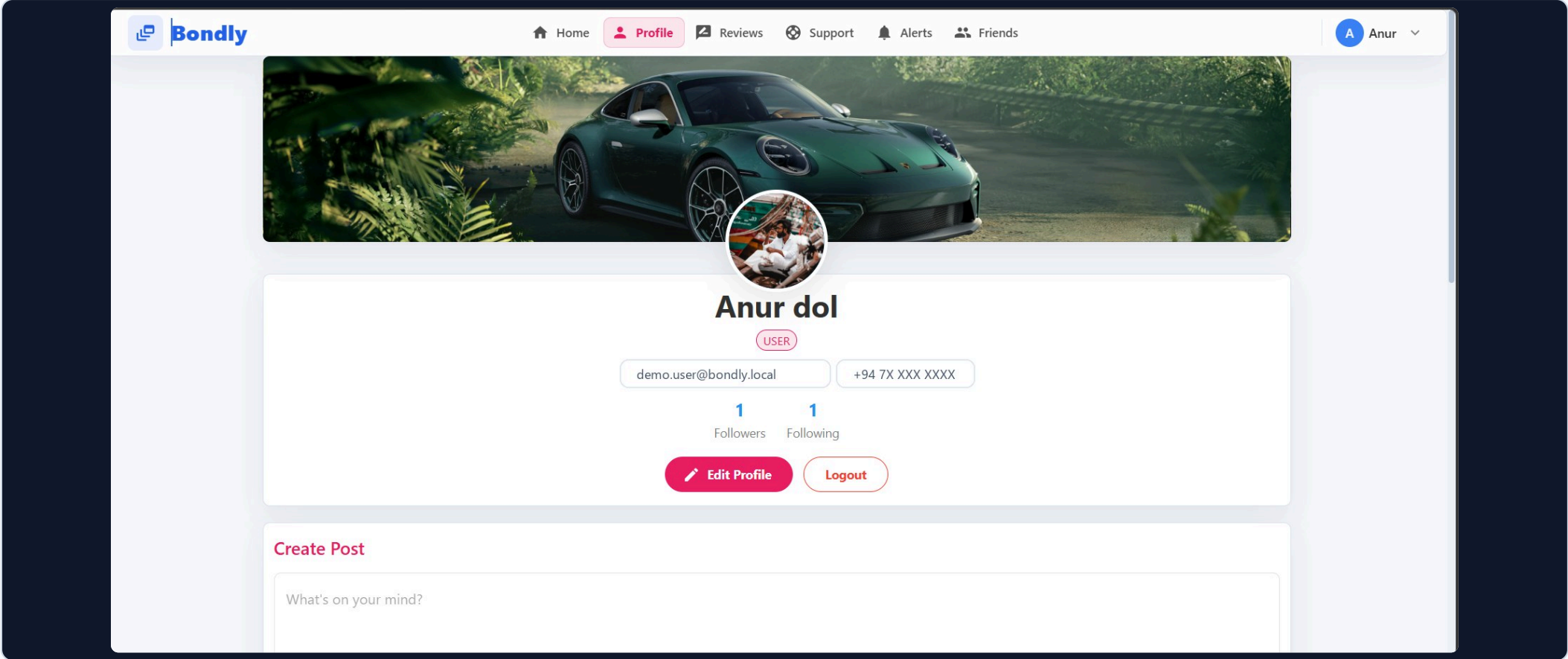
The same support module also has an admin response flow.

Support updates are wired through the realtime service for faster feedback.

04. Profile and Posting

UI walkthrough from the current Bondly interface.

User Identity



The profile page brings together cover image, profile image, role, follower counts, and post creation.

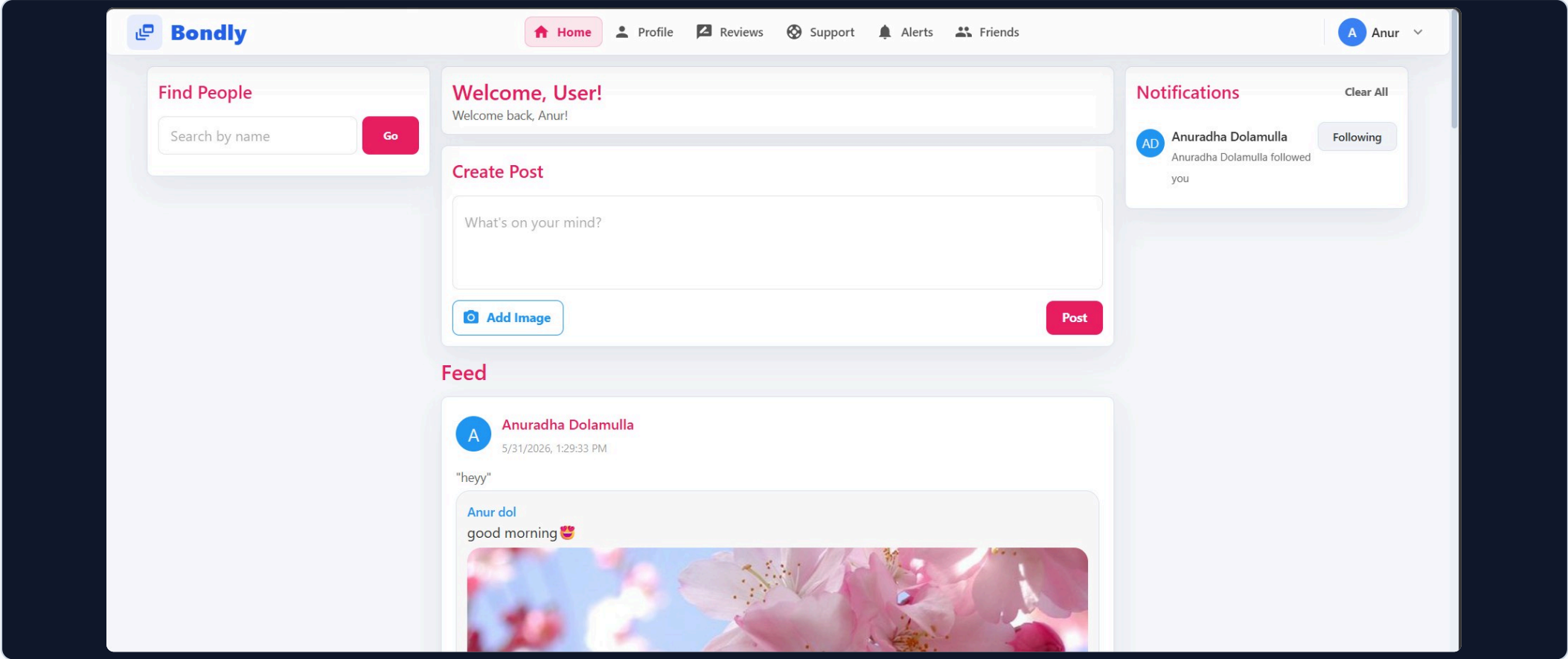
Profile and cover images are uploaded through multipart backend endpoints.

Public documentation uses masked contact details for safer sharing.

05. Home Feed

UI walkthrough from the current Bondly interface.

Core Feed UX



The home screen combines user search, post creation, feed content, and notifications.

Posts support text and image upload, with reaction/comment/share features handled by separate API modules.

The layout is built around repeated social workflows instead of a static landing page.

06. Registration Validation

UI walkthrough from the current Bondly interface.

Account Safety

Click to go back (Alt+Left arrow), hold to see history

Explore & Join

✓ Contact Details 3

Personal Info

Password *

Back Please fill out this field.

Please fill out this field.

The signup flow is split into personal info, contact details, and secure account steps.

Required fields protect the flow from incomplete account creation.

Backend validation and OTP verification complete the account creation process.

07. Contact Details Step

UI walkthrough from the current Bondly interface.

Recovery Data

Explore & Join 🌐

1 **Personal Info** 2 **Contact Details** 3 **Secure Account**

Email *

Temporary Email (Optional)

Phone Number *

[Back](#) [Next](#)

The account flow collects email, optional backup email, and phone number.

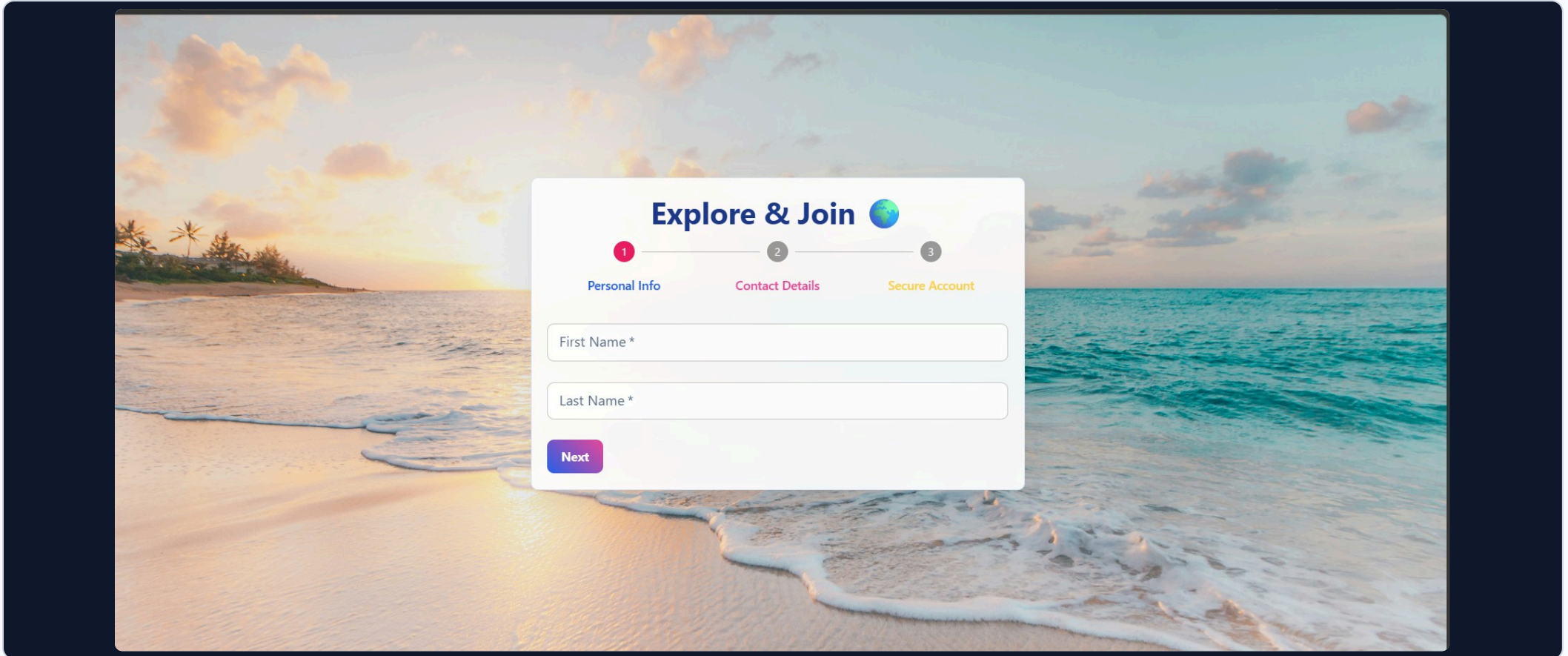
Forgot-password recovery can verify a user using multiple identity fields.

This supports a more practical recovery process than email-only reset.

08. Personal Details Step

UI walkthrough from the current Bondly interface.

Onboarding



Explore & Join 🌐

1 **Personal Info** 2 Contact Details 3 Secure Account

First Name *

Last Name *

Next

The first registration step keeps the initial user input simple.

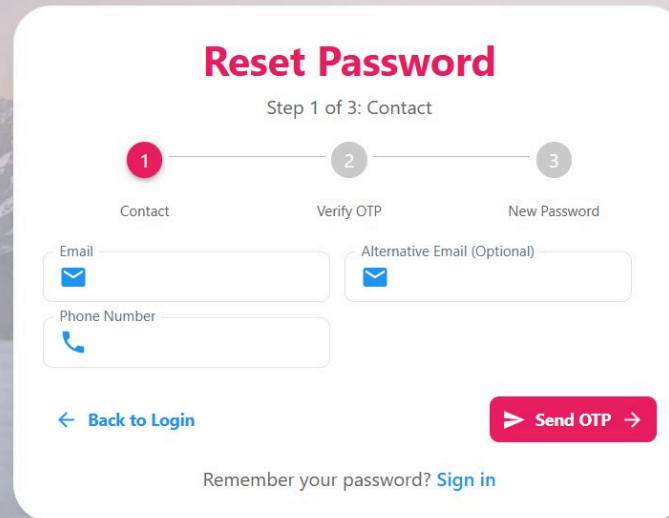
The progress indicator makes the multi-step flow easier to understand.

The page style matches the login and password recovery screens.

09. Forgot Password

UI walkthrough from the current Bondly interface.

OTP Recovery



Reset Password

Step 1 of 3: Contact

1 — 2 — 3

Contact — Verify OTP — New Password

Email 

Alternative Email (Optional) 

Phone Number 

[← Back to Login](#) [➤ Send OTP ➔](#)

Remember your password? [Sign in](#)

© 2026 Bondly. All rights reserved.

Password reset starts with contact verification and then moves through OTP and new password steps.

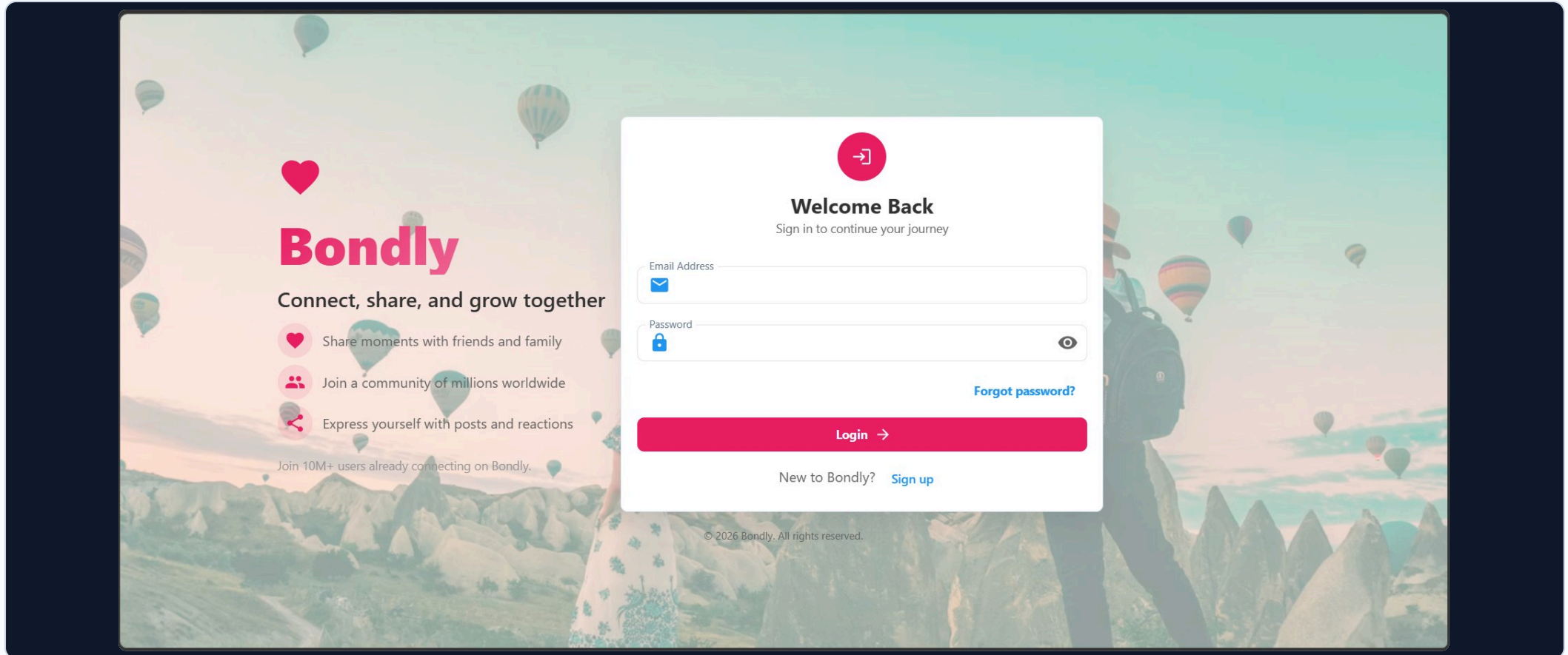
The backend limits resend attempts and blocks repeated OTP abuse for a period of time.

A short-lived verify token is used before allowing the password change.

10. Login Experience

UI walkthrough from the current Bondly interface.

Authentication



Login connects to the Spring Security authentication flow.

JWT access and refresh tokens are issued after successful authentication.

Role-based routing sends admin users and normal users to the right area.

Security and Repository Hygiene

What I keep safe in the current repo setup

The real application config should stay local. Database passwords, email app passwords, JWT secrets, uploaded files, crash logs, build outputs, node modules, and local environment files are ignored from new commits.

Real config: application.yaml

Public template: application.example.yaml

Runtime data: uploads/

Generated output: target/, dist/, node_modules/

Important note for a public repository

If a secret file was committed before it was added to .gitignore, the current commit can remove it from the latest repo state, but the old value can still exist in Git history. The correct fix is to rotate exposed credentials and then purge history with a dedicated cleanup step if the repository is public.

This PDF also masks personal contact data in screenshots so the project can be shared more safely.

Professional status

The repository is much stronger with a clear README, example configuration, ignored runtime files, and a project overview PDF. The remaining security decision is history cleanup: if old commits contain passwords or private uploads, they should be rotated and removed from history before treating the repository as fully clean.